

Python aplicat: fișiere, date și interfețe grafice

Paul A. Gagniuc

Cuprins

- ✓ Planul laboratorului
- ✓ Citire fișier TXT și extragere linii după etichetă
- ✓ Căi de fișiere, cwd, utf-8 și strip()
- ✓ Citire CSV: filtrare și calcule pe angajați
- ✓ DictReader: transformarea rândurilor în dicționare
- ✓ Scriere JSON
- ✓ Citire JSON
- ✓ Exemplu combinat: JSON + procesare
- ✓ Excel: citire, scriere și manipulare
- ✓ Generare date random și fișiere mari
- ✓ Procesare fără încărcare completă în memorie
- ✓ Trecerea de la OnlineGDB la lucrul local
- ✓ Introducere în GUI cu Tkinter
- ✓ Poziționarea obiectelor în interfață
- ✓ Asocierea obiectelor cu evenimente și operații
- ✓ Modificarea proprietăților obiectelor
- ✓ Validarea datelor introduse
- ✓ Citirea fișierelor text cu afișare în GUI
- ✓ Afișarea conținutului într-un TextBox cu scrollbar
- ✓ Reprezentarea grafică a datelor cu Matplotlib
- ✓ Grafice liniare, bare, histogramă și dispersie
- ✓ Citirea și reprezentarea grafică a datelor din CSV
- ✓ Integrarea graficelor Matplotlib în GUI
- ✓ Validarea datelor și tratarea erorilor
- ✓ Procesarea fișierelor CSV mari în bucăți
- ✓ Verificarea valorilor lipsă și a duplicatelor



Plan!

- A) Exerciții de scriere și citire a structurilor de date JSON și altele.
- B) Scriere și citire de fișiere Excel, manipulări de date.
- C) Generare de date randomizate și manipularea fișierelor mari.
- D) Continuare exemple plus exerciții.
- E) Începem lucrul cu interfețe grafice: basic, butoane, obiecte de input.
- F) Avem o discuție despre domeniul specific al fiecăruia, pentru a formula o temă de proiect individual, mai către sfârșitul cursului, dar este bine să discutăm din timp.

Citire fișier TXT și
extragere linii după
etichetă

TXT Example

Trecem de pe:

<https://www.onlinegdb.com/>

Task:

- ✓ numaram INFO/WARNING/ERROR
- ✓ extragem ERROR

Fisier: log.txt

TXT Code Example

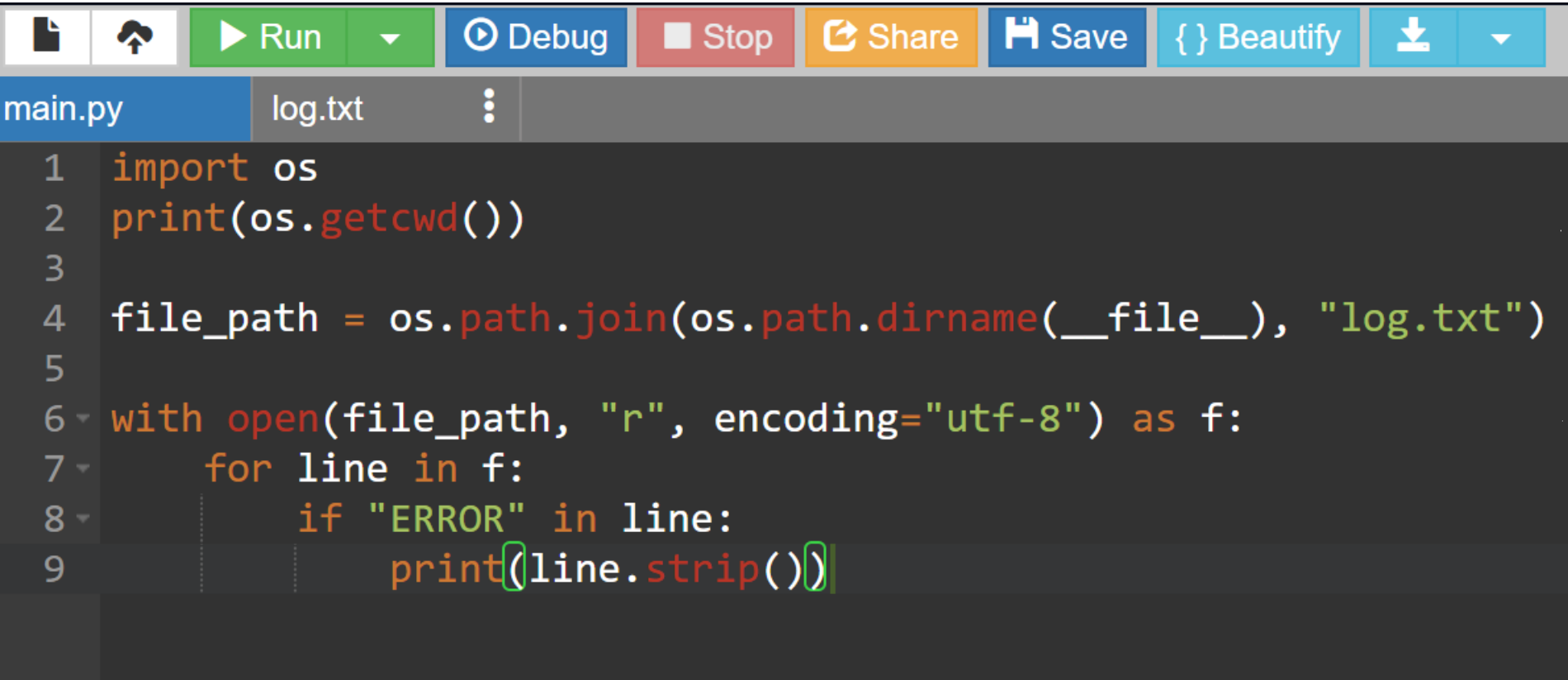
```
with open('log.txt','r') as f:  
    lines = f.readlines()
```

```
for line in lines:  
    if line.startswith('ERROR'):  
        print(line)
```

```
with open("C:/Users/Paul/Desktop/log.txt", "r", encoding="utf-8") as f:
```

```
file_path = os.path.join(os.path.dirname(__file__), "log.txt")
```

cwd = Current Working Directory



```
1 import os
2 print(os.getcwd())
3
4 file_path = os.path.join(os.path.dirname(__file__), "log.txt")
5
6 with open(file_path, "r", encoding="utf-8") as f:
7     for line in f:
8         if "ERROR" in line:
9             print(line.strip())
```



encoding="utf-8" = citește textul corect (cu diacritice, fără erori)

UTF = Unicode Transformation Format

strip() = șterge spațiile de la început și sfârșit

Citire CSV: filtrare și
calcul pe angajați

CSV

```
import csv

with open('angajati.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['nume'], row['credit'])
```

CSV = Comma-Separated Values

CSV este un fișier text care ține date în formă de tabel.

```
main.py log.txt angajati.csv
1 import os
2 import csv
3
4 cale = os.path.join(os.path.dirname(os.path.abspath(__file__)), "angajati.csv")
5
6 with open(cale, "r", encoding="utf-8") as f:
7     reader = csv.DictReader(f)
8     randuri = list(reader)
9
10 angajati_it = []
11 salariu_total = 0
12 salariu_maxim = -1
13 cel_mai_bine_platit = None
14
15 for rand in randuri:
16     salariu = int(rand["salariu"])
17     salariu_total += salariu
18
19     if rand["departament"] == "IT":
20         angajati_it.append(rand)
21
22     if salariu > salariu_maxim:
23         salariu_maxim = salariu
24         cel_mai_bine_platit = rand
25
26 print("Angajați IT:")
27 for ang in angajati_it:
28     print(ang["nume"], ang["salariu"])
29
30 print("Salariu mediu:", salariu_total / len(randuri))
31 print("Salariu maxim:", cel_mai_bine_platit["nume"], salariu_maxim)
```



DictReader = fiecare rând devine dicționar

```
["1", "Ana", "IT", "5500"]
```



```
{"id": "1", "nume": "Ana", "departament": "IT", "salariu": "5500"}
```

`list(reader)` = toate rândurile într-o listă





Scriere JSON

```
main.py  student.json ⋮
1  import json
2
3  student = {
4      "nume": "Popescu",
5      "varsta": 21,
6      "note": [9, 10, 8]
7  }
8
9  with open("student.json", "w") as f:
10     json.dump(student, f)
```

Scriere JSON



```
main.py  student.json ⋮
1  {"nume": "Popescu", "varsta": 21, "note": [9, 10, 8]}
```



Citire JSON

Citire JSON



```
main.py  student.json  ⋮
1  import json
2
3  with open("student.json", "r") as f:
4      data = json.load(f)
5
6  print(data["nume"])
7  print(data["note"])
```

✓ ↗ 📄 ⚙️ 📋

```
[9, 10, 8]
```

Exemplu combinat
(JSON + procesare)


```
main.py student.json ⋮
1 import json
2
3 with open("student.json", "r") as f:
4     data = json.load(f)
5
6 media = sum(data["note"]) / len(data["note"])
7 print("Media:", media)
```

input

Media: 9.0

Exemplu combinat (JSON + procesare)





Excel
citire, scriere,
manipulare

main.py

note.xlsx



```
1 import pandas as pd
2
3 # creare tabel
4 df = pd.DataFrame({
5     "nume": ["Ana", "Ion"],
6     "nota": [9, 7]
7 })
8
9 # salvare Excel
10 df.to_excel("note.xlsx", index=False)
11
12 # citire Excel
13 df2 = pd.read_excel("note.xlsx")
14
15 # manipulare
16 df2["nota"] = df2["nota"] + 1
17
18 print(df2)
```

```
pip install pandas
```

Excel
citire, scriere,
manipulare
(de testat local)

	nume	nota
0	Ana	10
1	Ion	8

```
import pandas as pd
print(pd.__version__)
```

Generare date
random + fișiere mari

```
main.py  date_mari.txt ⋮
1  import random
2
3  with open("date_mari.txt", "w") as f:
4      for i in range(100000): # fisier mare
5          numar = random.randint(1, 1000)
6          f.write(str(numar) + "\n")
```

Generare date
random + fișiere mari

```
main.py  date_mari.txt ⋮
21327  825
21328  980
21329  737
21330  449
21331  894
21332  479
21333  185
21334  925
21335  103
21336  881
21337  689
21338  868
21339  149
21340  932
21341  300
21342  611
21343  600
21344  918
21345  311
```

Procesare fără a
încărca tot în memorie

main.py

date_mari.txt :

```
1 total = 0
2
3 with open("date_mari.txt", "r") as f:
4     for linie in f:
5         total += int(linie)
6
7 print(total)
```

Procesare fără a încărca tot în
memorie

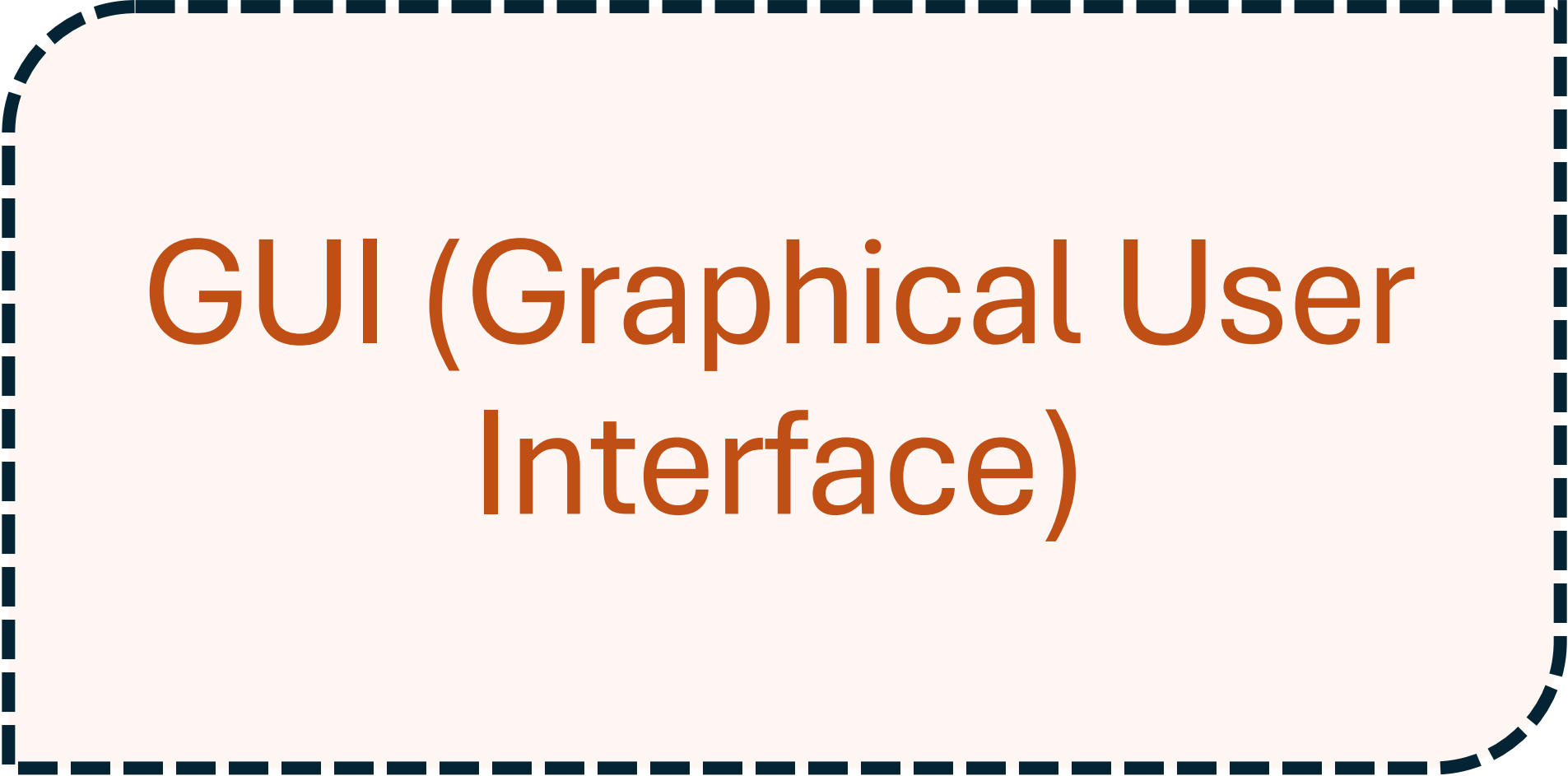


Schimbare pe local

Trecem de pe:

<https://www.onlinegdb.com/>

Pe varianta locală, unde instalăm Visual Studio.



GUI (Graphical User
Interface)

```
ex.py X
C: > Users > Paul > Desktop > ex.py > ...
1  import tkinter as tk
2
3  def afiseaza():
4      text = entry.get()
5      label.config(text="Ai scris: " + text)
6
7  root = tk.Tk()
8  root.title("Exemplu GUI")
9
10 entry = tk.Entry(root)
11 entry.pack()
12
13 buton = tk.Button(root, text="Apasa", command=afiseaza)
14 buton.pack()
15
16 label = tk.Label(root, text="")
17 label.pack()
18
19 root.mainloop()
```

root = fereastra principală a aplicației.

tk.Entry(root) = creează un câmp de text în fereastra root

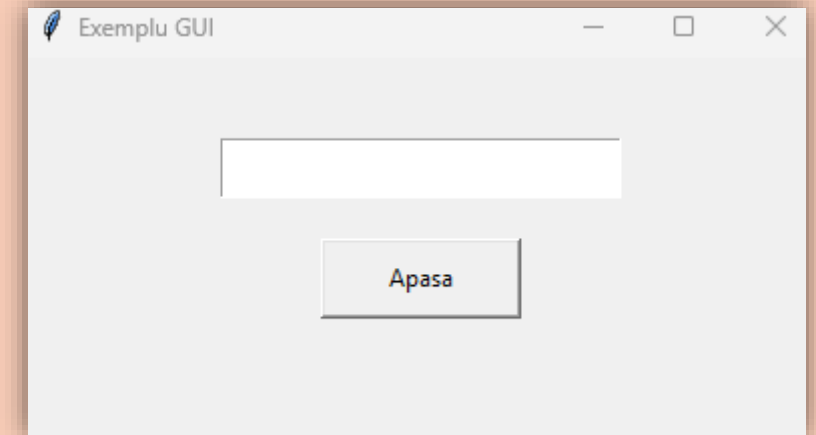
entry.pack() = spune unde să fie pus (îl afișează în fereastră)

Entry = input
root = unde apare
pack() = îl face vizibil

Poziționare exactă obiecte în Tkinter

```
ex.py
C: > Users > Paul > Desktop > ex.py > ...
1  import tkinter as tk
2
3  def afiseaza():
4      text = entry.get()
5      label.config(text="Ai scris: " + text)
6
7  root = tk.Tk()
8  root.title("Exemplu GUI")
9  root.geometry("400x250") # dimensiune fereastră
10
11  # Input
12  entry = tk.Entry(root)
13  entry.place(x=100, y=40, width=200, height=30)
14
15  # Buton
16  buton = tk.Button(root, text="Apasa", command=afiseaza)
17  buton.place(x=150, y=90, width=100, height=40)
18
19  # Label rezultat
20  label = tk.Label(root, text="")
21  label.place(x=100, y=150, width=200, height=30)
22
23  root.mainloop()
```

x, y → poziție
width, height → dimensiune
geometry() → dimensiunea ferestrei

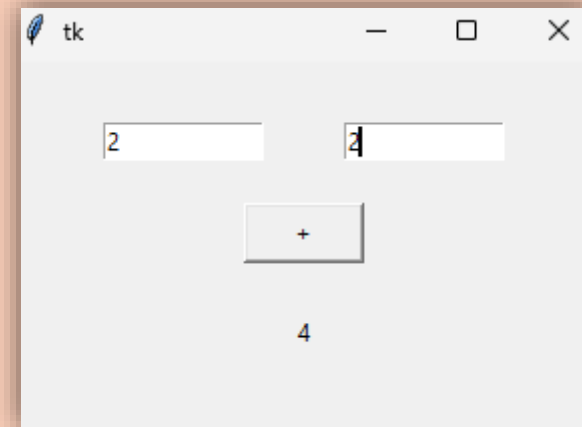


Dimensiuni exacte

Asocierea obiectelor
cu evenimente și
operații

```
ex.py X
C: > Users > Paul > Desktop > ex.py > ...
1  import tkinter as tk
2
3  def calc():
4      a = int(e1.get())
5      b = int(e2.get())
6      rez.config(text=str(a + b))
7
8  root = tk.Tk()
9  root.geometry("300x200")
10
11 e1 = tk.Entry(root)
12 e1.place(x=50, y=30, width=80)
13
14 e2 = tk.Entry(root)
15 e2.place(x=170, y=30, width=80)
16
17 btn = tk.Button(root, text="+", command=calc)
18 btn.place(x=120, y=70, width=60, height=30)
19
20 rez = tk.Label(root, text="")
21 rez.place(x=100, y=120, width=100, height=30)
22
23 root.mainloop()
```

Calculator

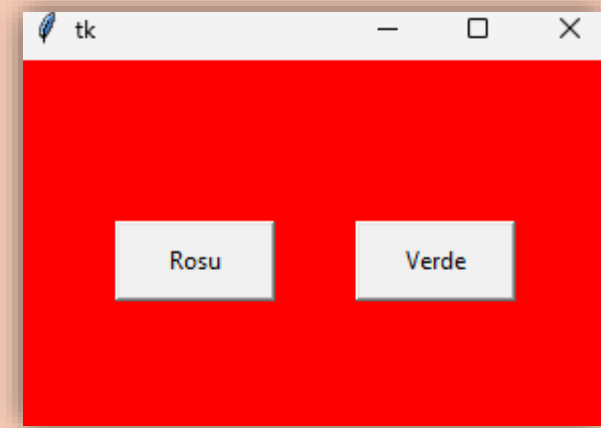




Modificarea proprietăților obiectelor

```
ex.py X
C: > Users > Paul > Desktop > ex.py > ...
1  import tkinter as tk
2
3  def rosu():
4      root.config(bg="red")
5
6  def verde():
7      root.config(bg="green")
8
9  root = tk.Tk()
10 root.geometry("300x200")
11
12 b1 = tk.Button(root, text="Rosu", command=rosu)
13 b1.place(x=50, y=80, width=80, height=40)
14
15 b2 = tk.Button(root, text="Verde", command=verde)
16 b2.place(x=170, y=80, width=80, height=40)
17
18 root.mainloop()
```

Schimbare culoare fundal



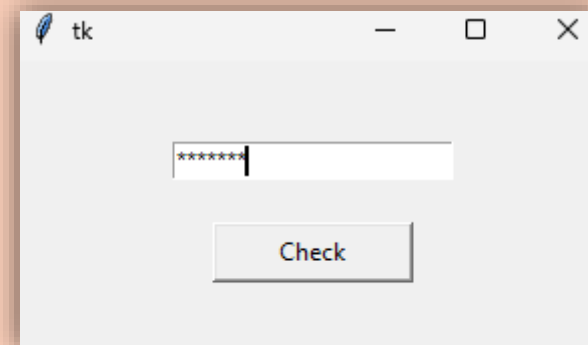


Validarea datelor
introduse

C: > Users > Paul > Desktop > ex.py > ...

```
1  import tkinter as tk
2
3  def verifica():
4      if entry.get() == "1234":
5          label.config(text="Corect")
6      else:
7          label.config(text="Gresit")
8
9  root = tk.Tk()
10 root.geometry("300x200")
11
12 entry = tk.Entry(root, show="*")
13 entry.place(x=80, y=40, width=140)
14
15 btn = tk.Button(root, text="Check", command=verifica)
16 btn.place(x=100, y=80, width=100, height=30)
17
18 label = tk.Label(root, text="")
19 label.place(x=100, y=130, width=100)
20
21 root.mainloop()
```

Verificare parolă



Citirea fișierelor text
cu afișare în GUI

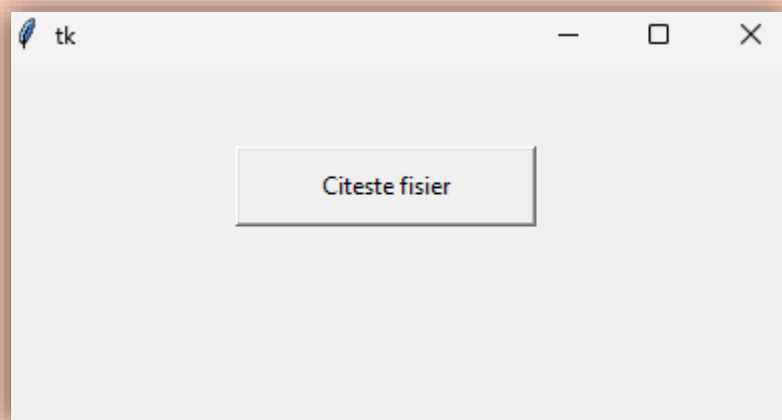
Citire fișier text

Slice:

`text[10:20]` # de la 10 la 20

`text[5:]` # de la 5 până la final

`text[-10:]` # ultimele 10 caractere

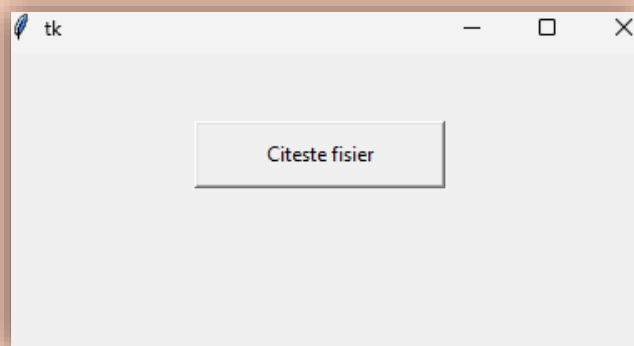


```
ex.py X
C: > Users > Paul > Desktop > ex.py > ...
1  import tkinter as tk
2
3  def citeste():
4      with open("test.txt", "r") as f:
5          text = f.read()
6          label.config(text=text[:50]) # primele 50 caractere
7
8  root = tk.Tk()
9  root.geometry("400x200")
10
11  btn = tk.Button(root, text="Citeste fisier", command=citeste)
12  btn.place(x=120, y=40, width=150, height=40)
13
14  label = tk.Label(root, text="")
15  label.place(x=50, y=120, width=300, height=40)
16
17  root.mainloop()
```

`text[:50]` afișează doar primele 50 caractere din fișier

Citirea fișierelor text
cu afișare în GUI prin
cale absolută

```
ex.py X
C: > Users > Paul > Desktop > ex.py > citeste
1 import tkinter as tk
2
3 def citeste():
4     with open(r"C:\Users\Paul\Desktop\ex.py", "r") as f:
5         text = f.read()
6         label.config(text=text[:50])
7
8 root = tk.Tk()
9 root.geometry("400x200")
10
11 btn = tk.Button(root, text="Citeste fisier", command=citeste)
12 btn.place(x=120, y=40, width=150, height=40)
13
14 label = tk.Label(root, text="")
15 label.place(x=50, y=120, width=300, height=40)
16
17 root.mainloop()
```

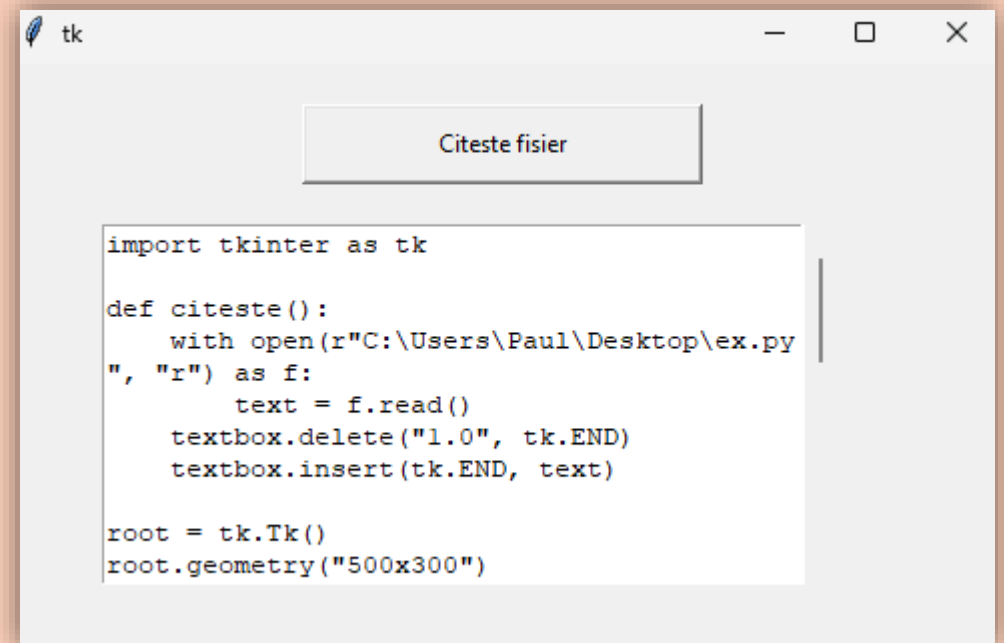


Cale absolută



Afișarea conținutului
unui fișier într-un
TextBox cu scrollbar

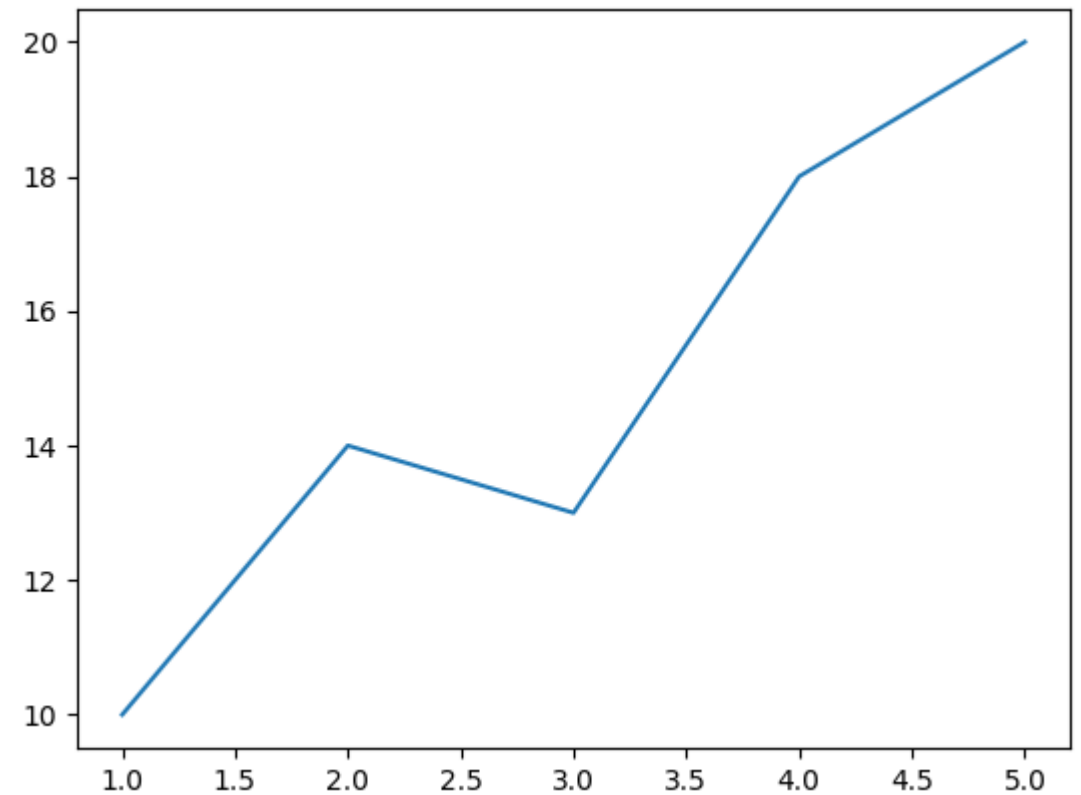
```
ex.py X
C: > Users > Paul > Desktop > ex.py > ...
1 import tkinter as tk
2
3 def citeste():
4     with open(r"C:\Users\Paul\Desktop\ex.py", "r") as f:
5         text = f.read()
6         textbox.delete("1.0", tk.END)
7         textbox.insert(tk.END, text)
8
9 root = tk.Tk()
10 root.geometry("500x300")
11
12 btn = tk.Button(root, text="Citeste fisier", command=citeste)
13 btn.place(x=150, y=20, width=200, height=40)
14
15 # Textbox
16 textbox = tk.Text(root)
17 textbox.place(x=50, y=80, width=350, height=180)
18
19 # Scrollbar vertical
20 scroll = tk.Scrollbar(root, command=textbox.yview)
21 scroll.place(x=400, y=80, height=180)
22
23 # Legătură Text ↔ Scroll
24 textbox.config(yscrollcommand=scroll.set)
25
26 root.mainloop()
```



Reprezentarea grafică a datelor cu Matplotlib

Reprezentarea grafică a datelor cu Matplotlib

```
C: > Users > Paul > Desktop > ex.py > ...  
1  import matplotlib.pyplot as plt  
2  
3  x = [1, 2, 3, 4, 5]  
4  y = [10, 14, 13, 18, 20]  
5  
6  plt.plot(x, y)  
7  plt.show()
```

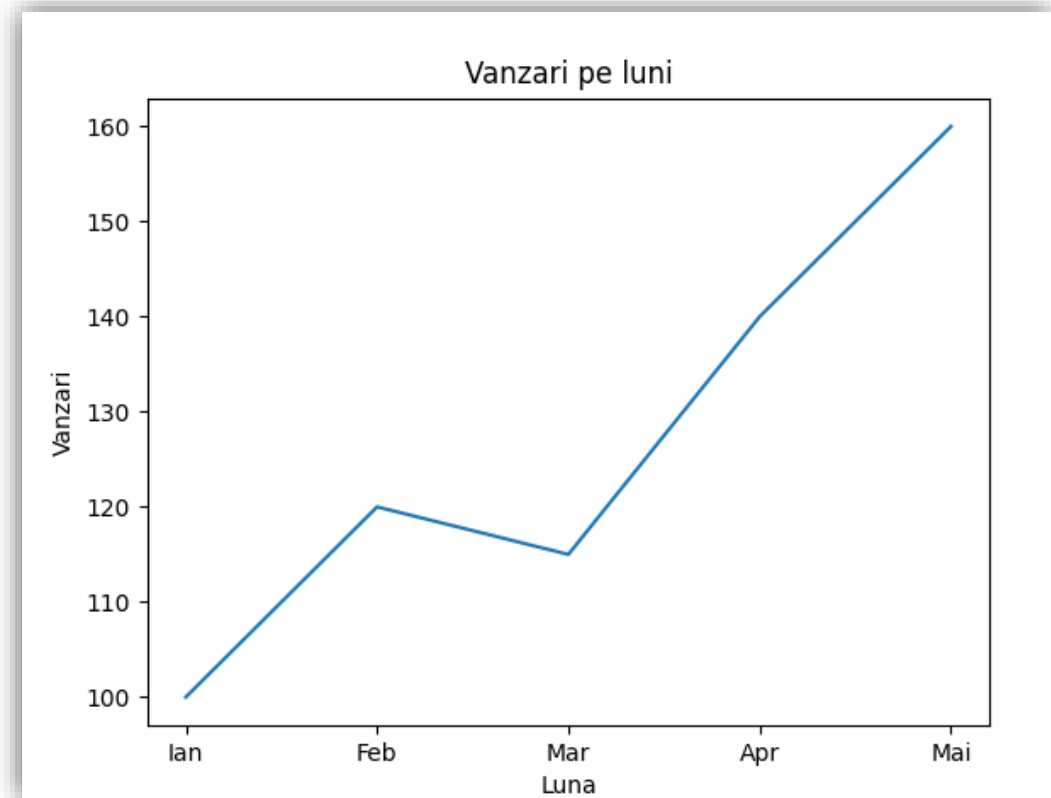


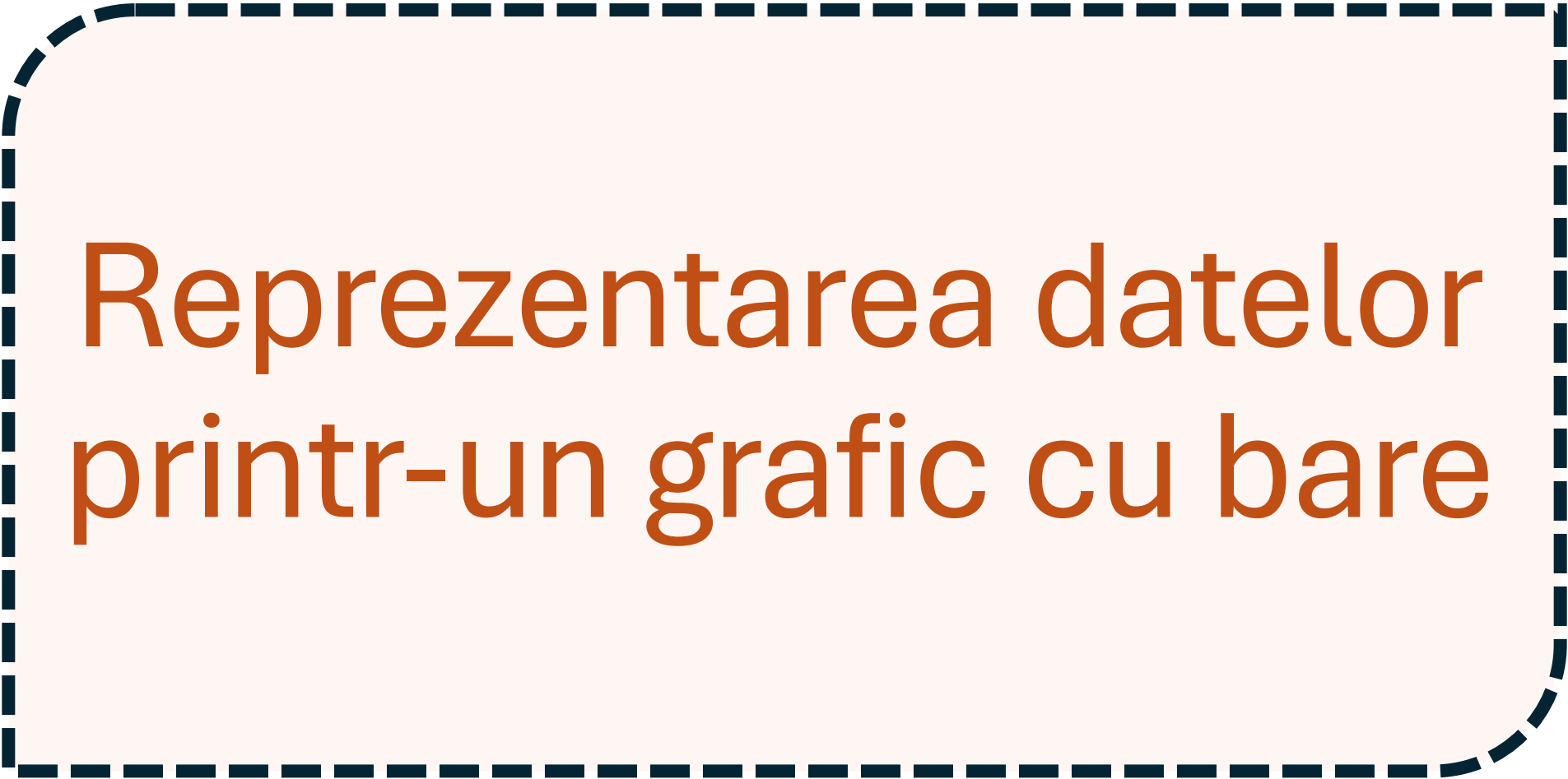
Adăugarea titlului și a
etichetelor graficului

Reprezentarea grafică a datelor cu Matplotlib

C: > Users > Paul > Desktop >  ex.py > ...

```
1  import matplotlib.pyplot as plt
2
3  luni = ["Ian", "Feb", "Mar", "Apr", "Mai"]
4  vanzari = [100, 120, 115, 140, 160]
5
6  plt.plot(luni, vanzari)
7  plt.xlabel("Luna")
8  plt.ylabel("Vanzari")
9  plt.title("Vanzari pe luni")
10 plt.show()
```



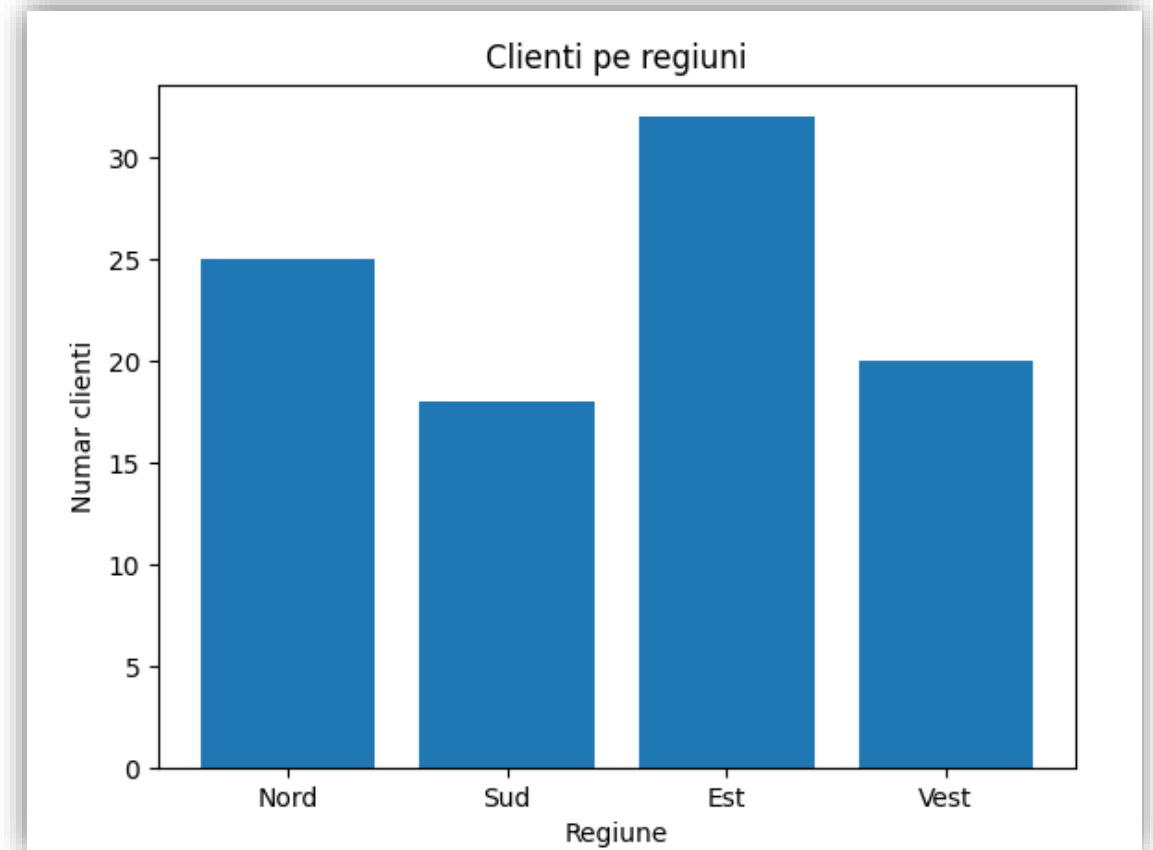


Reprezentarea datelor
printr-un grafic cu bare

C: > Users > Paul > Desktop > ex.py > ...

```
1  import matplotlib.pyplot as plt
2
3  regiuni = ["Nord", "Sud", "Est", "Vest"]
4  clienti = [25, 18, 32, 20]
5
6  plt.bar(regiuni, clienti)
7  plt.xlabel("Regiune")
8  plt.ylabel("Numar clienti")
9  plt.title("Clienti pe regiuni")
10 plt.show()
```

Reprezentarea grafică a datelor cu Matplotlib

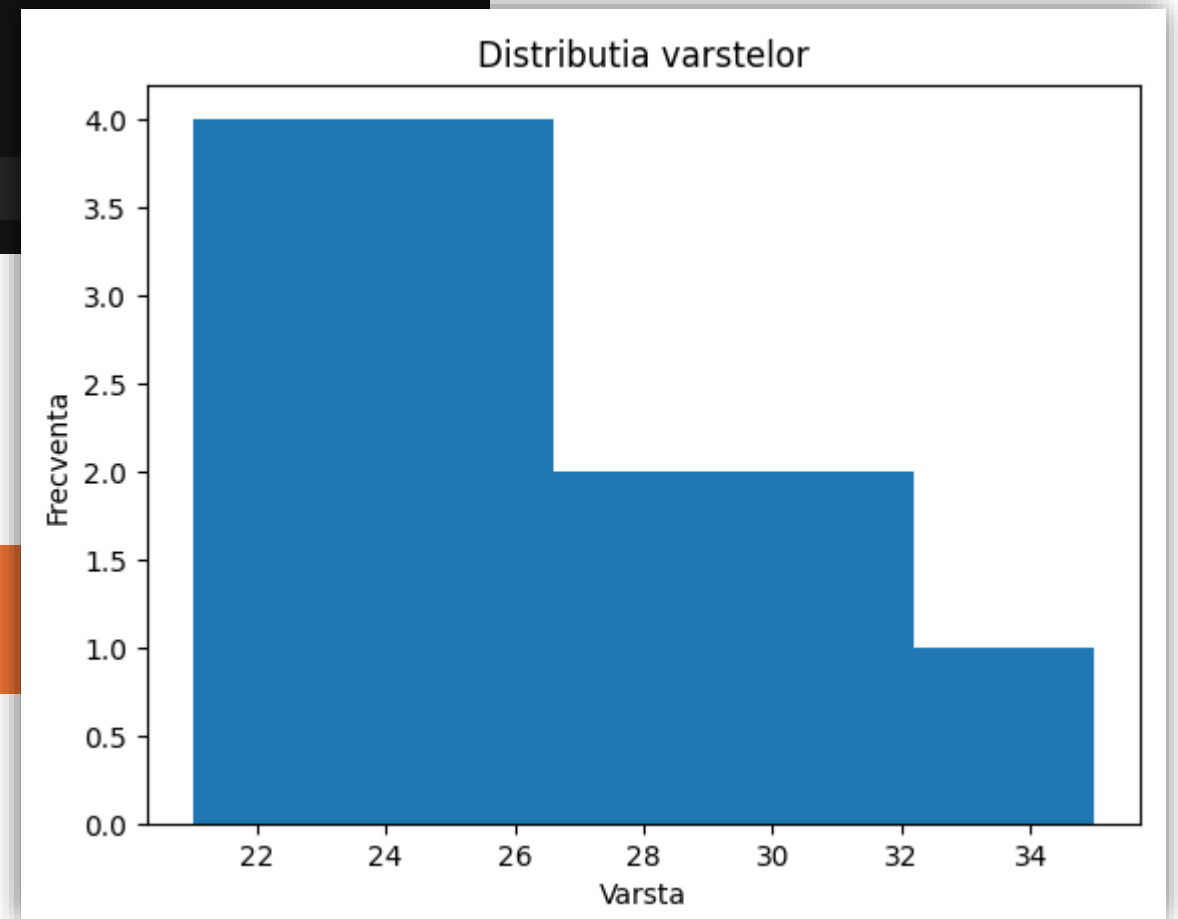


Reprezentarea distribuției datelor prin histogramă

C: > Users > Paul > Desktop > ex.py > ...

```
1 import matplotlib.pyplot as plt
2
3 varste = [21, 22, 22, 23, 24, 24, 25, 26, 27, 29, 30, 31, 35]
4
5 plt.hist(varste, bins=5)
6 plt.xlabel("Varsta")
7 plt.ylabel("Frecventa")
8 plt.title("Distributia varstelor")
9 plt.show()
```

Reprezentarea grafică a datelor cu Matplotlib

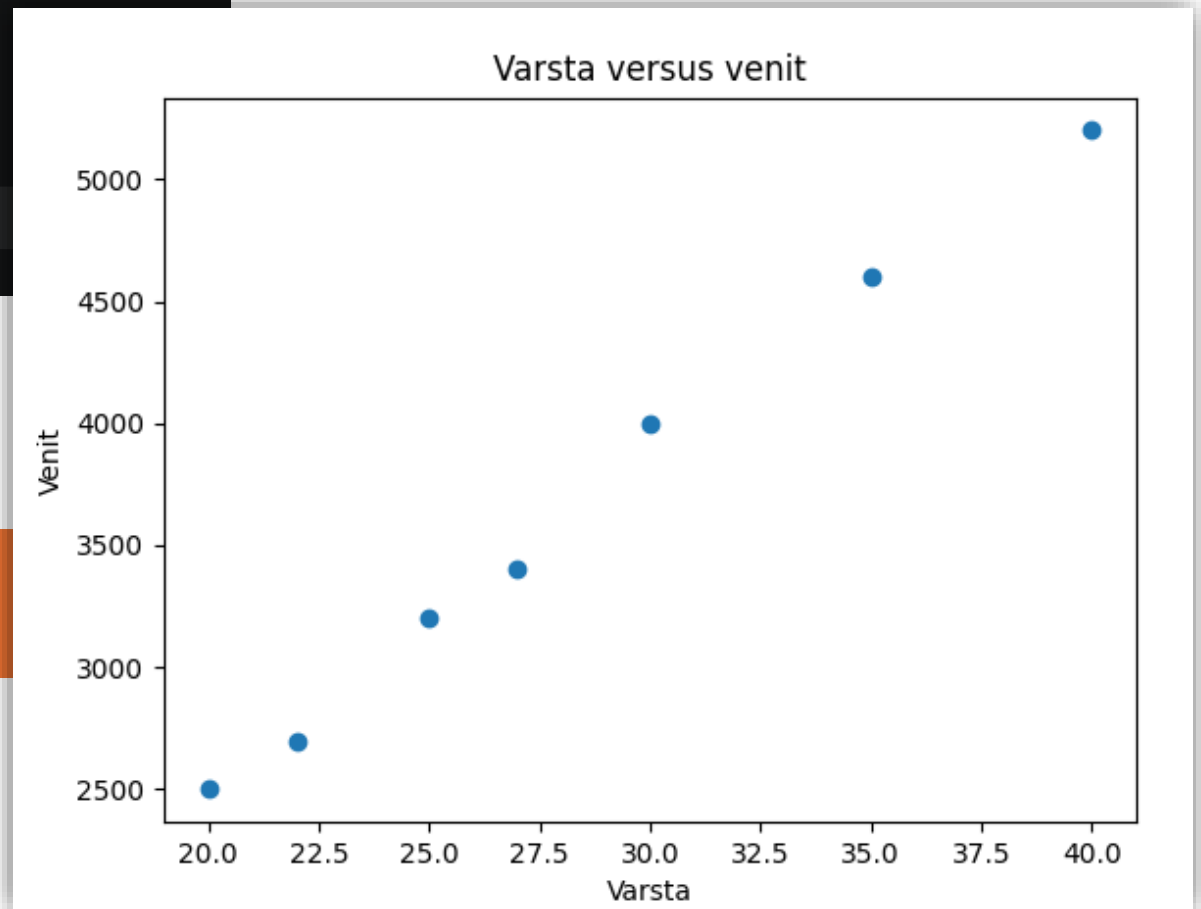


Reprezentarea relației
dintre variabile prin
diagramă de dispersie

C: > Users > Paul > Desktop > ex.py > ...

```
1  import matplotlib.pyplot as plt
2
3  varsta = [20, 22, 25, 27, 30, 35, 40]
4  venit = [2500, 2700, 3200, 3400, 4000, 4600, 5200]
5
6  plt.scatter(varsta, venit)
7  plt.xlabel("Varsta")
8  plt.ylabel("Venit")
9  plt.title("Varsta versus venit")
10 plt.show()
```

Reprezentarea grafică a datelor cu Matplotlib



Citirea și
reprezentarea grafică a
datelor din CSV

```
C: > Users > Paul > Desktop > ex.py > ...
```

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 df = pd.read_csv("vanzari.csv")
5
6 plt.plot(df["luna"], df["vanzari"])
7 plt.xlabel("Luna")
8 plt.ylabel("Vanzari")
9 plt.title("Vanzari din CSV")
10 plt.show()
```

Adaptati calea

Date din CSV + line plot
CSV vanzari.csv:

```
luna,vanzari
Ian,100
Feb,120
Mar,115
Apr,140
Mai,160
```



Integrarea graficelor Matplotlib în GUI

```
import tkinter as tk
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

def deseneaza_plot():
    ax.clear()
    ax.plot(x, y)
    ax.set_xlabel("X")
    ax.set_ylabel("Y")
    ax.set_title("Line plot")
    canvas.draw()

x = [1, 2, 3, 4, 5]
y = [10, 14, 13, 18, 20]

root = tk.Tk()
root.title("Plot in aceeași fereastră")
root.geometry("700x500")

buton = tk.Button(root, text="Afiseaza plot", command=deseneaza_plot)
buton.pack(pady=10)

fig = Figure(figsize=(6, 4), dpi=100)
ax = fig.add_subplot(111)

canvas = FigureCanvasTkAgg(fig, master=root)
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

root.mainloop()
```



Validarea datelor
introduse și tratarea
erorilor

```
import tkinter as tk
from tkinter import messagebox

def verifica():
    text = entry.get()

    try:
        valoare = float(text.replace(",", "."))
        label_rezultat.config(text=f"Valoare validă: {valoare}")
    except ValueError:
        messagebox.showerror("Eroare", "Introduceți un număr valid.")

root = tk.Tk()
root.title("Validare input")
root.geometry("300x150")

entry = tk.Entry(root)
entry.pack(pady=10)

button = tk.Button(root, text="Check", command=verifica)
button.pack()

label_rezultat = tk.Label(root, text="")
label_rezultat.pack(pady=10)

root.mainloop()
```



Procesarea fișierelor CSV mari în bucăți

```
import pandas as pd

fisier = "date_mari.csv"
total = 0

for bucata in pd.read_csv(fisier, chunksize=1000):
    total += bucata["valoare"].sum()

print("Total:", total)
```

Procesarea fișierelor CSV mari în bucăți



Verificare date lipsă și
duplicate

```
import pandas as pd

df = pd.read_csv("clienti.csv")

print("Valori lipsă:")
print(df.isna().sum())

print("Duplicate:")
print(df.duplicated().sum())

df_curat = df.dropna()
df_curat = df_curat.drop_duplicates()

print("Dimensiune inițială:", df.shape)
print("Dimensiune după curățare:", df_curat.shape)

df_curat.to_csv("clienti_curat.csv", index=False)
```



Exemplu naiv de
«criptare»: ROT13

Exemplu naiv de criptare: ROT13

```
import codecs

mesaj = "SALUT"

criptat = codecs.encode(mesaj, "rot_13")

print(criptat)

decriptat = codecs.decode(criptat, "rot_13")
```



Tehnic, ROT13 este o metodă de codificare, nu o criptare reală.

Criptarea mesajelor
client-server cu XOR

codare/decodare XOR

Client

C: > Users > Paul > Desktop > client.py > ...

```
1  import socket
2
3  mesaj = "Salut server"
4  cheie = 23
5
6  criptat = "".join(chr(ord(c) ^ cheie) for c in mesaj)
7
8  s = socket.socket()
9  s.connect(("127.0.0.1", 5000))
10
11 s.send(criptat.encode())
12
13 s.close()
```

Server

server.py X client.py Release Notes: 1.120.0

C: > Users > Paul > Desktop > server.py > ...

```
1  import socket
2
3  cheie = 23
4
5  s = socket.socket()
6  s.bind(("0.0.0.0", 5000))
7  s.listen(1)
8
9  client, addr = s.accept()
10
11 date = client.recv(1024).decode()
12
13 decriptat = "".join(chr(ord(c) ^ cheie) for c in date)
14
15 print("Mesaj primit:", decriptat)
16
17 client.close()
```


Generarea
executabilelor și
configurarea
client–server

```
pip install pyinstaller
```

```
pyinstaller --onefile server.py  
pyinstaller --onefile client.py
```

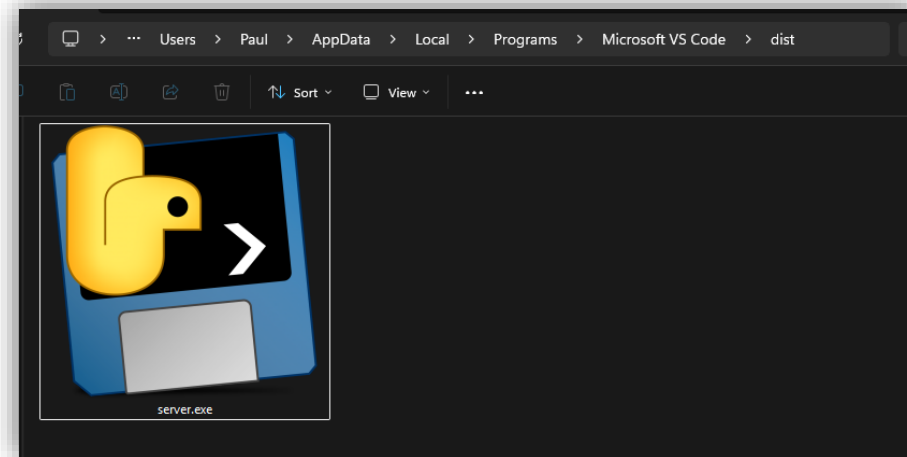
```
pyinstaller --onefile  
"C:\Users\Paul\Desktop\server.py"
```

Dacă sunt pe două calculatoare: clientul trebuie să știe IP-ul serverului.

```
s.connect(("192.168.1.50", 5000))
```



ipconfig



Apare folderul:

dist/



netstat -ano | findstr :5000

a = arată toate conexiunile și porturile
n = afișează numeric IP-ul și portul
o = arată PID-ul procesului

Daca portul este deja deschis:

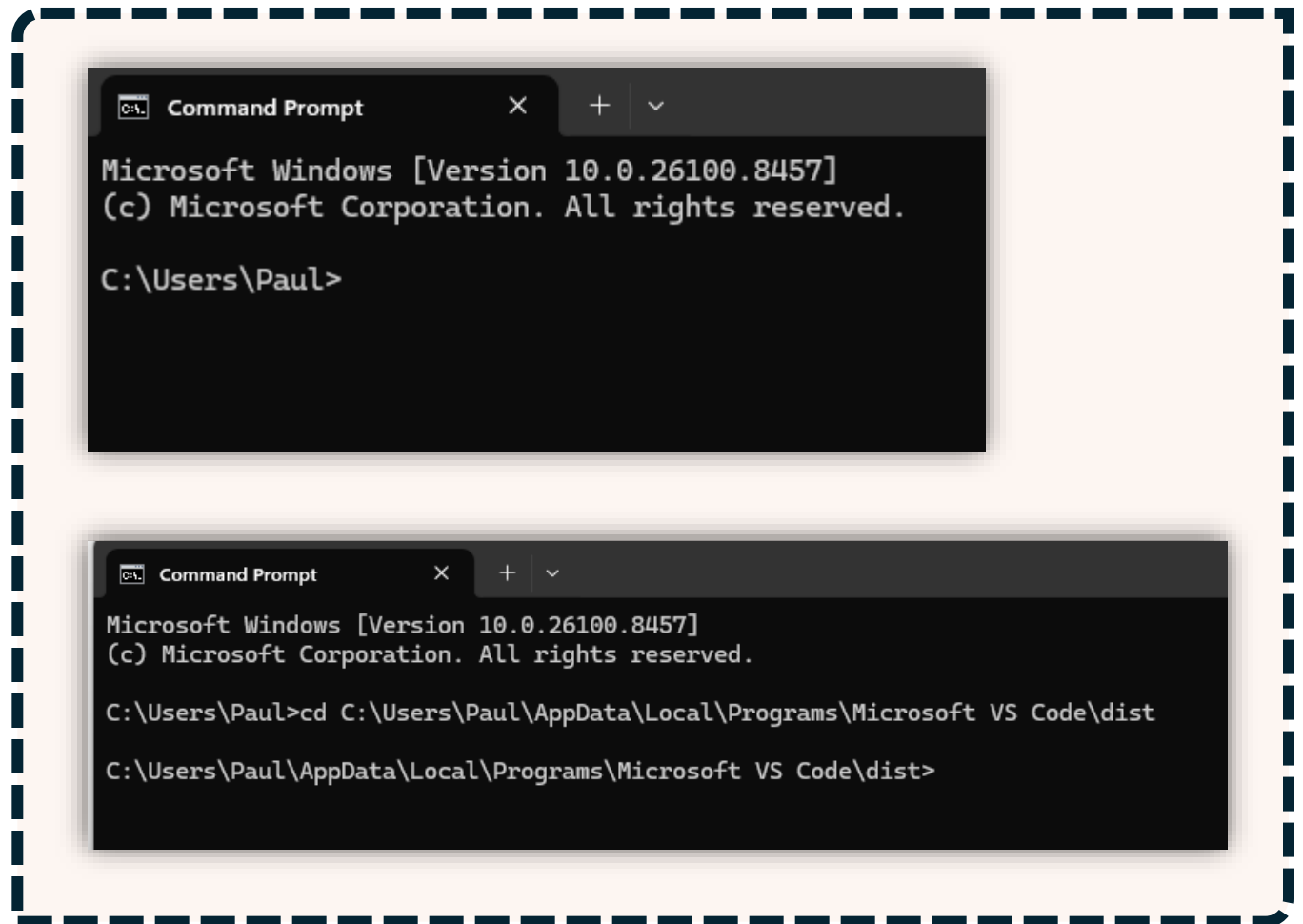
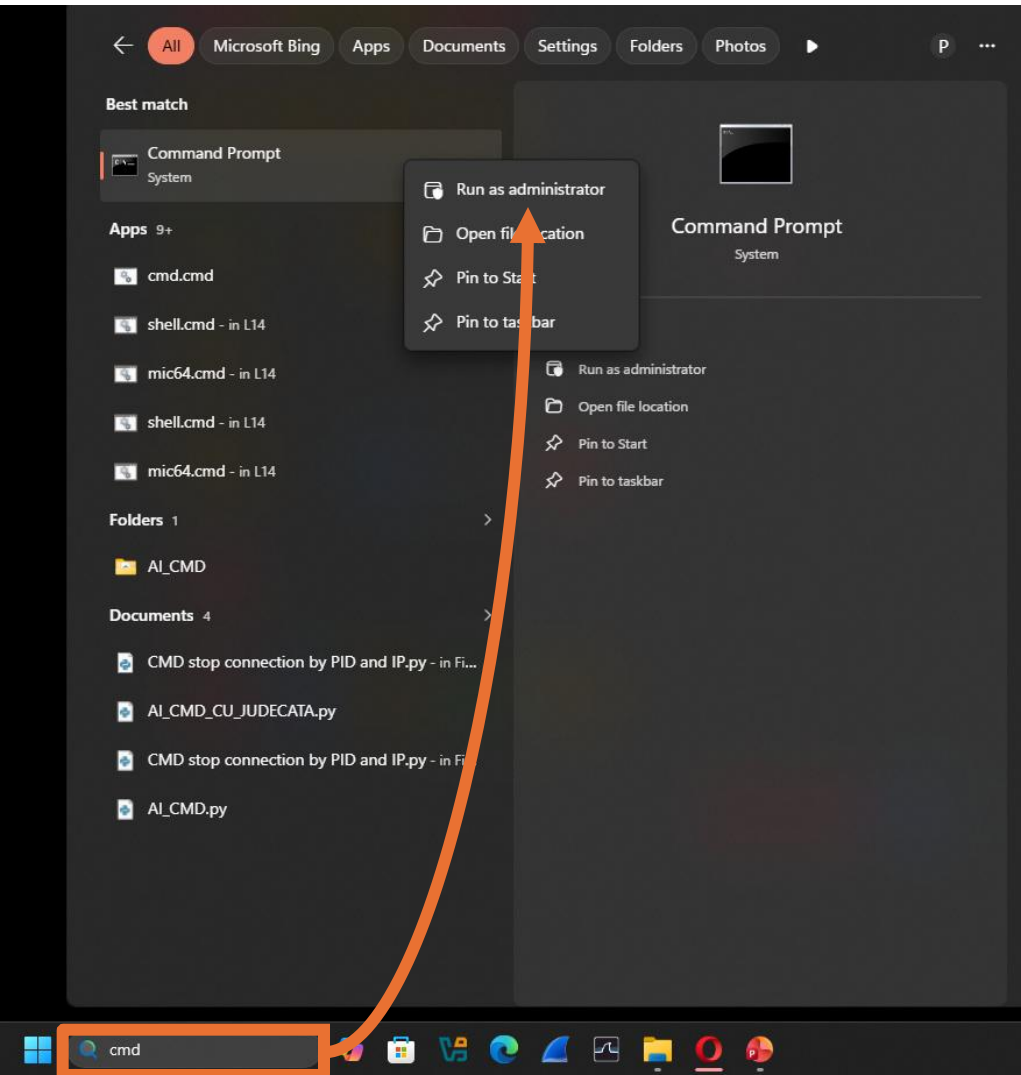
```
Command Prompt

C:\Users\Paul>cd C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code\dist

C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code\dist>server.exe
Traceback (most recent call last):
  File "server.py", line 6, in <module>
    OSError: [WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted
[PYI-8496:ERROR] Failed to execute script 'server' due to unhandled exception!

C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code\dist>
```

```
C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code\dist>server.exe  
Mesaj primit: Salut server  
  
C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code\dist>
```



Asteapta conexiunea

```
Command Prompt - server.ex  X  +  v
Microsoft Windows [Version 10.0.26100.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Paul>cd C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code\dist
C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code\dist>server.exe
|
```



Ză end!

